Software Documentation

1. Software Overview

- *What does the software do?*
- It manages the entire experience from the moment a player drops their name into the goblet right through to the trophy reveal ,handling sensor inputs, controlling all neopixel strips and servos, tracking game state, processing live Bluetooth data from a phone, and coordinating audio feedback through the buzzer
- *Platforms used (MicroPython, MIT App Inventor, etc.)*
- Platforms used are Microphyton, MIT app inventor, wokwi and Thonny

2. System Logic

Explain flow in simple steps

1. Idle state: blue Neopixel rotates pixel by pixel and bluetooth starts advertising continuously

2. IR sensor detects object: Neopixel changes from blue to red and Trial 1 starts

3. Trial 1: user presses button at correct distance range

   * If correct: servo opens and move forward

   * If wrong: red Neopixel indication and retry until max attempts

4. After passing Trial 1: system waits for LDR input

5. LDR detects darkness: Trial 2 starts

6. Trial 2: Neopixel sequence plays and user inputs matching pattern via Bluetooth

   * If correct: all Neopixels turn green and system waits for push button

   * If wrong: retry until max attempts

7. Push button pressed: Trial 3 starts

8. Trial 3: different colored Neopixels light up and user must match exact distance

   * If correct: one Neopixel turns off (1 point collected)

   * Repeat until 3 pixels are turned off

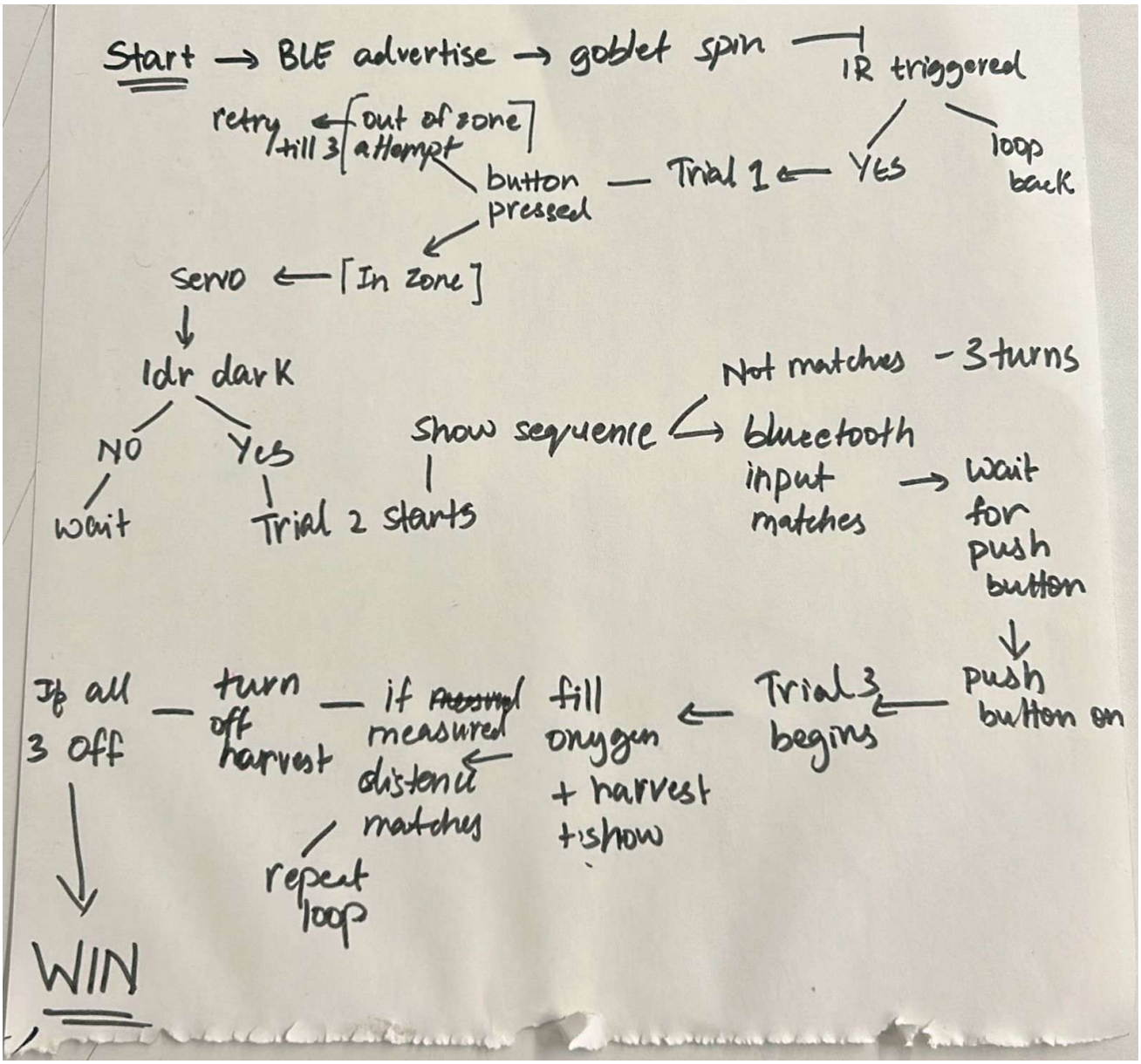
9. If 3 correct matches: trophy animation plays (win)

10. If wrong at any stage: retry until max attempts

11. If max attempts reached at any trial: game over (eliminated)

3. Flowchart

Visual representation of logic



4. App Flow (If MIT App Inventor)

- *Screen flow*

The app has a single scrollable screen with

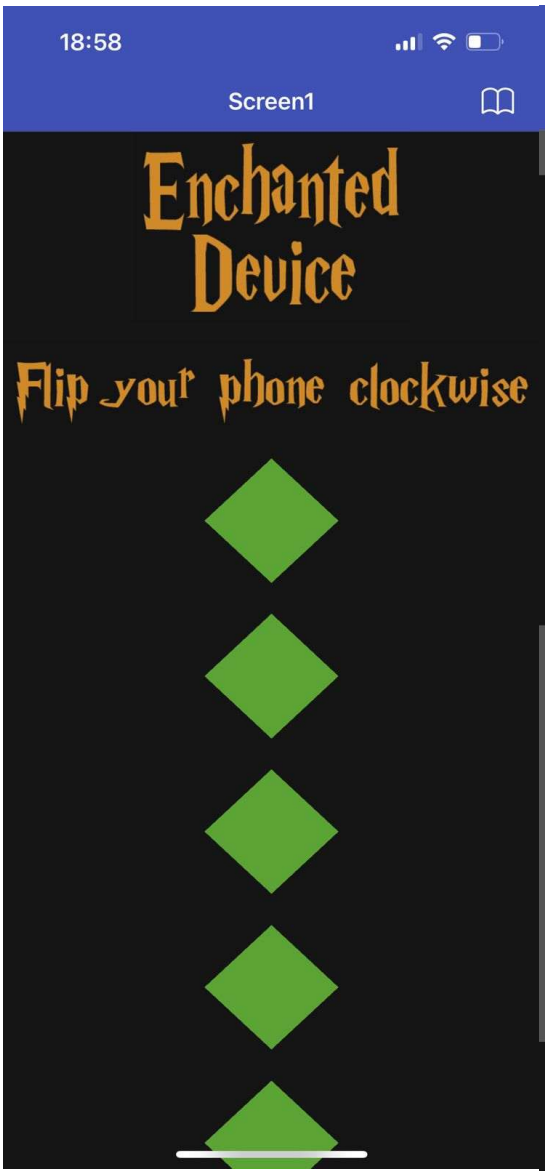
- A Bluetooth connection button
- A connection status label showing whether the phone is currently connected.
- Six large diamond shaped buttons labelled 1 through 6, vertically arranged. Each button corresponds to one of the six symbols in the sequence display.

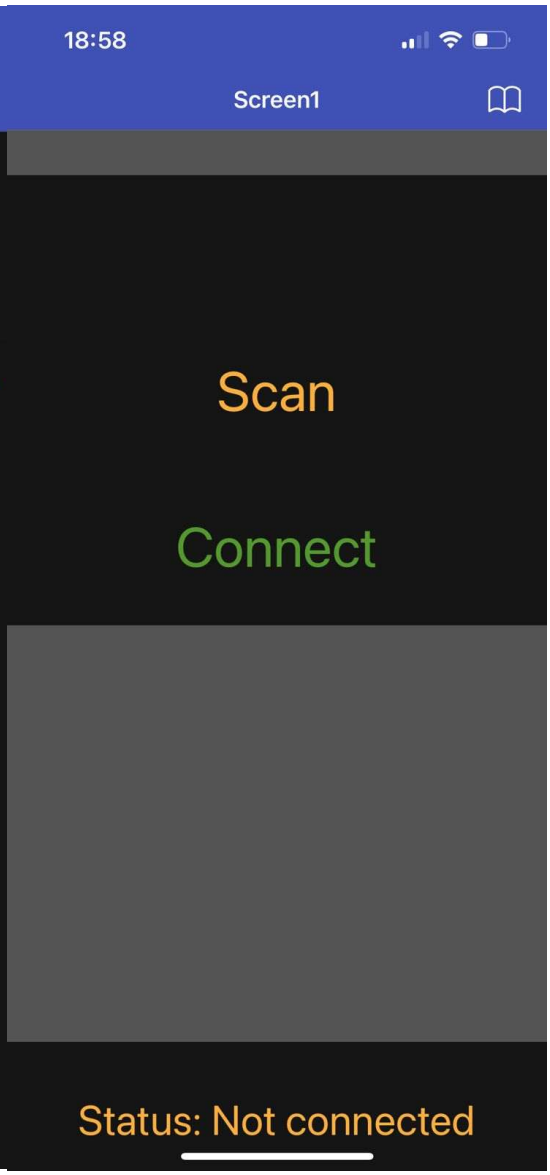
- *Button actions*
- Trial 2 requires 6 simultaneous input buttons. Wiring 6 physical buttons to the ESP32 would have consumed 6 GPIO pins plus required physical soldering of each button into the .MIT App Inventor moves this input to the player's phone over BLE, using 0 additional GPIO pins. The tradeoff is that theres a small delay sometimes but this was acceptable in trial 2 since there was no reaction time mechanism in this.
- Trial 1 and Trial 3 intentionally use physical buttons because those trials require real time physical interaction where delays would break the flow or lead to inaccurate results.

- *Data flow*

Player action	App behaviour	Data sent over BLE	ESP32 response
Tap connect button	App scans for BLE device named 'ESP32 hp', initiates connection	No data	Connects to esp 32
Tap button 1	BluetoothLE SendBytes component sends character '1' to the characteristic UUID	"1" (	ble_value set to "1", mapped to symbol index 0, appended to user_seq
Tap button 2	Sends character '2'	"2"	Mapped to symbol index 1
Tap button 3	Sends character '3'	"3"	Mapped to symbol index 2
Tap button 4	Sends character '4'	"4"	Mapped to symbol index 3
Tap button 5	Sends character '5'	"5"	Mapped to symbol index 4
Tap button 6	Sends character '6'	"6"	Mapped to symbol index 5
Phone disconnects	App disconnects from the esp32	No data	Restarts automatically

-


18:58
Screen1
Enchanted Device
Flip your phone clockwise


18:58
Screen1
Scan
Connect
Status: Not connected